

Take-Home Test - QA Automation Engineer

eDOT | Version 4 | Deadline: 3 days after you receive this assignment

This test checks whether you can design test cases that actually verify behaviour, automate them on the stack eDOT uses today (**Playwright** for web, **Maestro** for mobile, **Python + Pytest + Allure** on both), and put **AI inside the suite** with sensible guardrails. We grade judgement, not volume - a small suite that genuinely verifies behaviour beats a large one that only proves the pages loaded.

Ground rules

Python + Pytest only. Selenium and Appium are not accepted.
Page Object Model for web; reusable sub-flows (runFlow) for mobile.
No sleep(). Use Playwright auto-waiting / Maestro wait commands.
Allure reporting on both platforms. No credentials or API keys in the repo.
You must be able to explain every line you submit.

01 Phase 1 - Manual Test Case Design

Design the test cases **before** writing any script. Submit as Google Sheet or Excel.

Scope

Web (eSuite)	1. Login to eSuite • 2. Create a new company • 3. Verify the created company's data matches the input in the detail view
Mobile (eWork SFA)	4. Login using the newly created company credentials • 5. Create a customer

Required columns

Test Case ID | Title / Description | Precondition | Test Steps | Test Data (exact values) | Expected Result | Assertion Tier | Status (leave blank)

Assertion tiers - graded on both the sheet and the code

Tier 1	Navigation / display only. Asserting the page or element is visible is enough.
Tier 2	Any test that creates, edits or deletes data. Assert the outcome and the data itself : the record exists and its values (name, email, phone, address, ...) equal what you entered. A success toast alone is not sufficient. For a delete, assert the record is gone; for an edit, assert the value changed.
Negative	Assert the specific error message, not merely that the form is still open.

Mark Tier 2 assertions in code with a short comment, so a reviewer can tell a product bug from a script defect at a glance.

02 Phase 2 - Automation Implementation

2A. Web - Playwright + Pytest

Target	https://esuite.edot.id
Credentials	Email it.qa@edot.id / Password it.QA2025
Login flow	Three screens: click "Use Email or Username", submit the email, then submit the password. You are redirected through the eDOT Account Center and back - budget for this in your page object.

Scenarios

- **Login** - assert the dashboard greeting ("Welcome Back,") is displayed.
- **Create company** - Companies → "+ Add Company" opens a 3-step Register Company wizard. Step 1 needs Company Name, Email, Phone, Industry Type, Company Type, Language, Street Address and the dependent cascade Country → Province → City → District → Zone → Postal Code. Next stays disabled until the step is valid. Use AI-generated dummy data (Phase 3A).

- **Verify detail (Tier 2)** - open it via Companies → Manage and assert field by field: name, industry type, company type, address, postal code, email, phone.
- **Clean up** - delete the company at the end of the run. This is a shared environment; leftover data counts against you.

Engineering requirements

- Locators live in page classes - no raw selectors in test files. Priority: data-testid → role + accessible name → stable attribute (name / id / aria-*) → text as a last resort (justify it in a comment).
- No `time.sleep()`. Use auto-waiting and `expect()`.
- Log in once per session and share auth via `storage_state` - do not log in inside every test.
- Attach a screenshot on failure to the Allure report.

2B. Mobile - Maestro + Pytest

App	ework - SFA (Google Play Store)
Prerequisites	Maestro CLI (macOS / Linux, or Windows via WSL) - verify with <code>maestro --version</code> . An emulator or device visible to <code>adb devices</code> . Use <code>maestro studio</code> or <code>maestro hierarchy</code> for selectors.
Credentials	Prefer the company you created in 2A - a new company gets a 30-day trial, so it is active. Fallback (may be expired): Company ID 5049209, user <code>salesmanqaauto</code> , password <code>it.QA2025</code> . If you use the fallback, say so in the README.

Scenarios

- **Login** - assert the dashboard is displayed.
- **Create customer** - input the data and assert it appears correctly after creation (Tier 2).

Engineering requirements

- Flows are YAML; a Pytest wrapper invokes them so web and mobile report into the same Allure run.
- Extract anything reused - login especially - into a shared sub-flow referenced with `runFlow`.
- Pass credentials and test data in as environment variables. Never hardcode them in YAML.
- Selector priority: id → id regex → text → accessibilityText → composite → point. A coordinate tap is a last resort and must be justified in a comment.
- No sleep-based waiting - use Maestro wait commands. Attach the Maestro output, and a screen recording where possible, to the Allure step.

03 Phase 3 - AI Inside the Automation

This is what separates V4 from a conventional automation exercise: the AI must live **in the suite**, not just in your editor while you write it. Both capabilities below are mandatory.

3A. AI-generated test data

- A module that asks a model for coherent, realistic Indonesian business data - company (legal name, email, phone, street address, industry) and customer (name, contact, address).
- **Validate the output against a schema** before any test consumes it. Reject and retry, or fall back, on malformed output.
- **Deterministic fallback** (e.g. Faker) used automatically when no API key is present, so the suite still runs offline and in CI.
- Attach the data actually used to the Allure report.

3B. AI failure triage

A script that runs after the suite, reads the Allure results, and for each failure returns a verdict: **script/environment defect**, **product bug**, or **flaky**. Walk the evidence in this order and stop at the first match:

- Exception (element not found, timeout) or failed assertion? An exception is almost always script or environment, not a bug.
- Did the locator resolve to the intended, unique element?
- Did every step before the assertion succeed, and were the preconditions met?

- Was the expected value itself correct according to the test case?
- Does it reproduce consistently, or only sometimes? Intermittent means flaky, not a bug.

Output a triage report (Markdown or HTML) giving the verdict and the evidence behind it for every failure.

Guardrails (graded)

AI must never weaken, skip or rewrite an assertion, swallow a failure in try/except, or change an expected value to match the actual result. A failing test stays failing.

The triage verdict is a proposal for a human. It must not auto-file bugs and must not auto-close anything.

The API key comes from an environment variable and is never committed. Be deliberate about token cost.

AI_USAGE.md - document at minimum

- Which model you used and why.
- Where AI runs: while writing the tests, during the run, or after the run.
- The exact prompts you send.
- What happens when the AI is unavailable or returns something invalid.
- What you deliberately did **not** let AI do, and why.

04 Deliverables

Test case document	Google Sheet or Excel covering web and mobile, with your GitHub repository link inside it.
GitHub repository	Clean modular structure • web code (Playwright + Pytest) • mobile code (Maestro flows + Pytest wrappers) • both AI modules • Allure setup • README.md (dependencies, playwright install, Maestro CLI, emulator/adb, how to run each suite, how to generate and open the Allure report) • AI_USAGE.md .
Evidence of execution	An Allure report from at least one full web run, plus a triage report from a deliberately failing run - break something on purpose (e.g. point a locator at the wrong element) so we can see the triage works.

05 How This Is Graded

All three phases are graded. The two things that move the needle most are the **depth of your assertions** (Tier 2 done properly across web and mobile) and the **AI implementation** - both capabilities working, with schema validation and a working fallback. Allure and documentation are graded on whether a reviewer can actually follow what you did, not on volume.

We would rather see three scenarios that genuinely verify behaviour, with an honest note about the fourth you could not finish, than five that all pass because the assertions cannot fail.

Non-negotiables

Credentials or API keys committed to the repository.

A green suite achieved by weakening assertions, changing expected values, or skipping tests.

Test data left behind on the shared environment.

Being unable to explain code you submitted - it will be treated as not your work.

Bonus (optional)

- A CI pipeline (e.g. GitHub Actions) that runs the web suite headless and publishes the Allure report.
- Parallel execution.
- A genuine data handoff between platforms: the company created by the web suite is the one the mobile suite logs into.

Submission: 3 days after receiving this assignment. Send the GitHub repository link together with your manual test case sheet.